User Defined Functions Notes

18 July 2025 14:22

Definition:

A user-defined function is a function created by the **user** to perform a specific task.
You can use it **whenever you want**, and you can also **change its internal code** (modify the task).

Q. Why use user-defined functions?

- ✧ To **reuse code** (write once, use many times)
- ✧ To **divide large programs** into smaller, readable parts
- ✧ To **make code modular and manageable**

Q. How can we create user function?

Syntax: def Function_name(arguments):

```
    Instructions or statements
    return values
print( Function_name(values))
```

- ❖ **def —> Function Definition Keyword**

  Explanation:
  - def is a **keyword** in Python used to define a **user-defined function**.
  - It tells Python: "Hey, I'm about to create a function.
- ❖ **Function_name —> Function Identifier**

  Explanation:
  - This is the **name of the function** that you create.
  - You will use this name when you want to **call the function**.
  - It should follow variable naming rules: lowercase, no spaces, no special symbols.
- ❖ **return —> Send Back a Value**

  Explanation:
  - return is used to **give a result back** from a function to where it was called.
  - It ends the function and **returns** a value.
  - Without return, the function just runs but does not give back a result
- ❖ **print() —> Display the Result**

  Explanation:
  - print() is a **built-in Python function** used to **display output** on the screen.
  - If your function returns something, you use print() to show it.
- ❖ **Arguments in Function Declaration**

  These are also called "**parameters**".

  What are they?
  - These are **variable names** used inside the function.
  - They are written inside parentheses after the function name during **definition**.
  - Used to receive input values when the function is called.

  Example:def greet(name): # 'name' is the argument (parameter)
  print("Hello", name)

  Note:"Here, name is the argument used inside the function to receive the value."
- ❖ **Values in Function Calling**

  These are also called **input values**.

  What are they?
  - These are the **real values** you pass when calling the function.
  - These values go into the function and replace the parameters.

Example:

```
def add(a, b):        # a, b → arguments (parameters)
    return a + b

print(add(10, 20))    # 10, 20 → values (actual arguments)
```

Note: in above example:

- a receives 10, and b receives 20
- Function adds them → 10 + 20 = 30
- Output: 30

In Python, **user-defined functions** can behave differently based on two things:

1. **Whether they take inputs (arguments)**
2. **Whether they give back outputs (return values)**

Based on these, we can classify functions into **4 types**.

Types Of user defined Fyunctions:

1.function without arguments and without return values.
2.function with arguments and without return values.
3.function without arguments and with return values.
4.function with arguments and with return values .

1.function without arguments and without return values(or)No Arguments, No Return Value:

- The function **does not take input** and **does not return** anything.
- It only performs an action (like printing something).

Syntax: def **Function_name():**

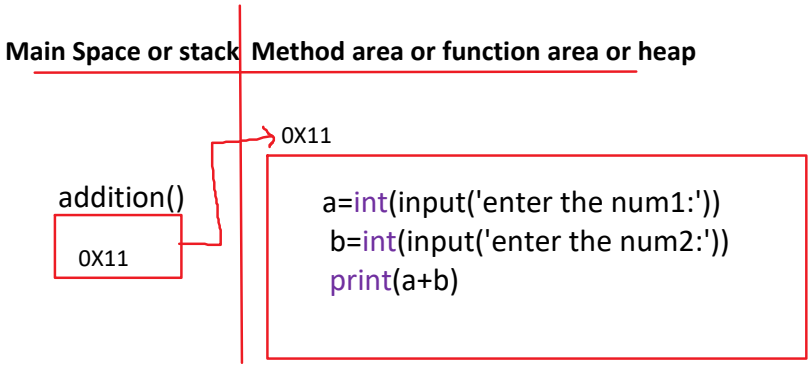
```
    Instructions or statements

print( Function_name())
```

Examples:

1.Wap to find the sum of two 2 number

```
def addition():
    a=int(input('enter the num1:'))
    b=int(input('enter the num2:'))
    print(a+b)
addition()
```



Output: enter the num1:10
enter the num2:20
30

2.wap to count the number of occurance of a given character in a string

```
#example:'Functionsss'-->'s'-->3

    def count_char():
      s=input('enter the string:')
      ch=input('enter the character:')
      count=0
      for i in s:
        if i==ch:
          count=count+1
      print(count)
  count_char()
```

Output: enter the string:functionssss
enter the character:s
4

#Assignment--->18-07-25

1.wap to convert the string to uppercase

```
    def convert_up():
        s = input('Enter the string: ')
        out = ''
        for i in s:
            if 'a'<=i<='z':
                out += chr(ord(i)-32)
            else:
                out += i
        print(out)
    convert_up()
```

Output: enter the string:Python
PYTHON

2.Wap to extract string from the list which are pallindrome
#example:l=['hi',10,20,30,'mom','dad']

```
    def extract_palindromes():
      words = ['hi',10,20,30,'mom','dad']
      print("Original list:", words)
      print("Palindrome strings in the list are:")

      for word in words:
        if word == word[::-1]:  # Check if the word is same as its reverse
          print(word)
    extract_palindromes()
```

Output: Original list: ['hi',10,20,30,'mom','dad']
Palindrome strings in the list are:
Mom
dad

3.wap program to find the greater among 3 numbers

```
def find_greatest():
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    c = int(input("Enter third number: "))

    if a >= b and a >= c:
        print("Greatest number is:", a)
    elif b >= a and b >= c:
        print("Greatest number is:", b)
    else:
        print("Greatest number is:", c)

find_greatest()
```

Output: Enter first number: 10
Enter second number:20
Enter third number:30
Greatest number is:30

4.wap to concatenate two list collections with out using + operator

```
'''l1=[1,2,3]
l2=[4,5,6]
l1+l2===>[1,2,3,4,5,6]'''
def concatenate():
    l1=[1,2,3]
    l2=[4,5,6]
    out=l1
    for i in l2:
        out.append(i)
    print(out)
concatenate()
```

Output: [1, 2, 3, 4, 5, 6]

#5.wap to perform all operation(arithmetic,relational,logical.....) by using function

2.function with arguments and without return values or no return values:

- The function **does take input** and **does not return** anything.

Syntax: def Function_name(arguments):

print(Function_name(values))

1.Wap to find sum of 2 numbers

```
def add(a,b):
    print(a+b)
add(10,20)
```

2. WAP to convert the string to uppercase.

```
def convert_up(s,out):
    for i in s:
        if 'a'<=i<='z':
            out += chr(ord(i)-32)
        else: out += i
            print(out)
    convert_up(input('Enter the string: '))
```

3. WAP to count the number of occurence of character in a string.

```
def count_char(s,ch,count):
    for i in s:
        if i == ch:
            count += 1
    print(count)
count_char(input('Enter the string: '),input('Enter the character: '),0)
```

3.function without arguments and with return values or no arguments:

- The function **doesn't take input** and **does return** anything.

Syntax: **def Function_name():**

print(Function_name())

1.Wap program to perform arithmetic operators

```
def Arithmetic_operations():
   a=int(input('enter the number1:'))
   b=int(input('enter the number2:'))
   return a+b,a-b,a*b,a/b,a//b,a%b,a**b

 add=Arithmatic_operations()
print(add)
```

2.Example :

```
def student_details():
   return 'krishnaveni','trainer'

name,role=student_details()
print(name,role)
```

3.wap program to find the sum of integers present in a given set(by using 3rd case)

```
def sum_int():
    s=eval(input('enter the set :'))
    Sum=0
    for i in s:
        if type(i)==int:
            Sum+=i
    return Sum
print(sum_int())
```

4.function with arguments and with return values :

- The function **does take input** and **does return** anything.

Syntax: def Function_name(arguments):

print(Function_name(values))

Example1:

```
def opration(a,b):
    return a>b,a<b,a==b,a!=b

print(opration(10,20))
```

2.wap to map two list collection in the form of dictionary

```
'''l1=['a','b','c']
l2=[10,20,30]
Output:{'a':10,'b':20,'c':30}'''

def map_list(l1,l2):
    out={}
    if len(l1)==len(l2):
        for i in range(0,len(l1)):
            out[l1[i]]=l2[i]
        return out
    else:
        return 'both list have a different length'
print(map_list(['a','b','c'],[10,20,30]))
```

#Assignment--->19-07-25

#1.Write a program to print initial index of a character in a given string

```
#example:-'programming'--->'m'---->initial index of 'm'
def get_initial_index(s, ch):
    if ch in s:
        return s.index(ch)  # Returns the first index of character
    else:
        return -1
print(get_initial_index(input('enter the string:'),input('enter the character:')))
```

Output: enter the string:python programming
enter the character:o
4

```
#2.wap to extract all the negative integer numbers from the list
def extract_negatives(num):
    out = []
    for i in num:
        if i < 0:
            out.append(i)
    return out

print(extract_negatives(eval(input('enter the list:'))))
```

Output:
enter the list:[10,20,-10,-5.5]
[-10, -5.5]